অধ্যায় ৩

Operators, Expressions এবং Decision Making

শেখার লক্ষ্য

এই অধ্যায়ের শেষে পাঠক JavaScript-এর Arithmetic, Assignment, Comparison ও Logical Operators ব্যবহার করতে পারবে; `if` / `else`, `switch`, Ternary এবং Guard Clause দিয়ে সিদ্ধান্ত নিতে পারবে; Type Coercion-এর ঝুঁকি বুঝবে; এবং একটি **Shopping Discount Calculator** তৈরি করবে।

৩.১ Expression, Statement ও Operator

Expression এমন কোড যা একটি Value তৈরি করে: `5 + 3`, `price > 1000`, `user?.name`। **Statement** কোনো কাজ নির্দেশ করে: `if (...) { ... }`, `const total = ...;`। Operator এক বা একাধিক Operand নিয়ে কাজ করে।

```javascript
const unitPrice = 850;
const quantity = 3;
const subtotal = unitPrice * quantity; // 2550
console.log(subtotal >= 2000); // true
```

`unitPrice * quantity` একটি Expression; `const subtotal = ...` একটি Declaration Statement। Expressions-এর Precedence বোঝা জরুরি; সন্দেহ হলে Parentheses ব্যবহার করো।

৩.২ Arithmetic Operators

Operator	কাজ	উদাহরণ	ফলাফল
`+`	যোগ	`7 + 3`	`10`
`-`	বিয়োগ	`7 - 3`	`4`
`*`	গুণ	`7 * 3`	`21`
`/`	ভাগ	`7 / 2`	`3.5`
`%`	Remainder	`7 % 3`	`1`
`**`	Power	`2 ** 3`	`8`

```javascript
console.log(10 / 0);        // Infinity
console.log(0 / 0);         // NaN
console.log(-7 % 3);        // -1 (remainder; mathematical modulo নয়)
console.log(0.1 + 0.2);     // 0.30000000000000004
```

JavaScript-এর `number` হলো IEEE 754 Double-Precision Floating Point। টাকা-পয়সার হিসাবের ক্ষেত্রে সরাসরি Fractional Decimal যোগ না করে Integer Minor Units (যেমন পয়সা) বা উপযুক্ত Decimal Library ব্যবহার করা ভালো। এই Project-এ BDT-এর পূর্ণসংখ্যা Value ব্যবহার করব।

৩.৩ Assignment, Increment ও Decrement

```javascript
let stock = 10;
stock += 5; // 15
stock -= 2; // 13
stock *= 2; // 26
stock /= 2; // 13
stock++;    // 14
--stock;    // 13
```

`x++` আগে পুরোনো Value দেয়, তারপর বাড়ায়; `++x` আগে বাড়ায়, তারপর নতুন Value দেয়। Complex Expression-এর মধ্যে এদের ব্যবহার করলে Readability কমতে পারে।

৩.৪ Comparison: `===` বনাম `==`

```javascript
console.log(5 === '5');  // false: Type ও Value উভয়ই মিলতে হবে
console.log(5 == '5');   // true: Coercion ঘটে
console.log(5 !== '5');  // true
console.log(10 > 5);     // true
console.log(10 <= 10);   // true
```

সাধারণত `===` ও `!==` ব্যবহার করো। `NaN === NaN` false; `Number.isNaN(value)` দিয়ে যাচাই করো। Object Comparison Reference-Based:

```javascript
console.log({ id: 1 } === { id: 1 }); // false
const a = { id: 1 };
const b = a;
console.log(a === b); // true
```

৩.৫ Logical Operators ও Short-Circuiting

`&&` (AND), `||` (OR), `!` (NOT) Boolean Logic-এ ব্যবহৃত হয়, তবে `&&` ও `||` সবসময় Boolean Return করে না—Operand-এর Value Return করে।

```javascript
const isMember = true;
const total = 3000;
console.log(isMember && total >= 2000); // true

const displayName = '' || 'Guest'; // 'Guest'
const seats = 0 ?? 10;             // 0
const unknownSeats = null ?? 10;   // 10
```

`||` বনাম `??`: `||` যেকোনো Falsy Value (`0`, `''`, `false`) বাদ দেয়; `??` শুধু `null` ও `undefined`-এর ক্ষেত্রে Fallback দেয়। `??`-কে `||` বা `&&`-এর সঙ্গে একই Expression-এ ব্যবহার করলে Parentheses প্রয়োজন।

৩.৬ Operator Precedence এবং Parentheses

```javascript
console.log(2 + 3 * 4);   // 14
console.log((2 + 3) * 4); // 20
const eligible = (age >= 18) && (hasId === true);
```

Precedence মুখস্থ করার চেয়ে Intent পরিষ্কার করে Parentheses ব্যবহার করা ভালো। Assignment-এর সঙ্গে Comparison মিশিয়ে জটিল One-Liner এড়িয়ে চলো।

৩.৭ `if`, `else if`, `else`

```javascript
function shippingMessage(amount) {
  if (!Number.isFinite(amount) || amount < 0) {
    return 'Invalid amount';
  }
  if (amount >= 5000) {
    return 'Free shipping';
  } else if (amount >= 2000) {
    return 'Discounted shipping';
  }
  return 'Standard shipping';
}
```

এখানে শেষের `return` হলো **Guard/Default Flow**। Nested `if` অনেক হলে Function ভেঙে ছোট করা যায়।

৩.৮ `switch` Statement

```javascript
function paymentLabel(method) {
  switch (method) {
    case 'card': return 'Card payment';
    case 'cash': return 'Cash on delivery';
    case 'mobile': return 'Mobile banking';
    default: return 'Unknown payment method';
  }
}
```

`switch` Strict Comparison (`===`-এর মতো) ব্যবহার করে। `break` বা `return` না দিলে পরবর্তী Case-এ Fallthrough হতে পারে। Intentional Fallthrough ছাড়া এটি এড়িয়ে চলো।

৩.৯ Ternary ও Guard Clauses

```javascript
const badge = stock > 0 ? 'In stock' : 'Sold out';

function validateQuantity(quantity) {
  if (!Number.isInteger(quantity) || quantity <= 0) {
    return { ok: false, error: 'Quantity must be a positive integer' };
  }
  return { ok: true };
}
```

Simple Two-Way Decision-এর জন্য Ternary ভালো; জটিল Nested Ternary পড়া কঠিন।

৩.১০ Common Pitfalls

1. `=` হলো Assignment; `===` হলো Comparison।
2. `Number('')` ফল `0`; Blank Input-কে সরাসরি Number করলে ভুল Validation হতে পারে।
3. `parseInt('12abc', 10)` ফল `12`; কঠোর Numeric Validation-এর জন্য এটি যথেষ্ট নয়।
4. `0` Falsy হলেও একটি বৈধ Quantity বা Price হতে পারে।
5. Percentage Discount যোগ করার ক্রম Business Rule অনুযায়ী নির্ধারণ করতে হবে।
6. Client-Side Discount Calculator শুধু UI Preview; বাস্তব Checkout-এ Server-Side Price Recalculation জরুরি।

৩.১১ Practical Project — Shopping Discount Calculator

Business Rules (এই শিক্ষামূলক Project-এর জন্য):

- Item Price: পূর্ণসংখ্যা BDT, `0` বা বেশি।
- Quantity: ১–১০০-এর মধ্যে পূর্ণসংখ্যা।

- Coupon: `SAVE10` দিলে ১০% Discount; `SAVE20` দিলে ২০% Discount, তবে Subtotal কমপক্ষে BDT 5,000 হতে হবে।
- Member Discount: Coupon-এর পরে অবশিষ্ট Amount-এর ওপর ৫%।
- Delivery: Discounted Total BDT 3,000 বা বেশি হলে Free; অন্যথায় BDT 120।
- Percentage Discount প্রতিটি ধাপে নিকটতম পূর্ণ BDT-তে Round করা হবে।
- Unknown Coupon-এ Error দেখানো হবে; Silent Ignore নয়।

Project Structure

```
shopping-discount-calculator/
├── index.html
├── style.css
├── calculator.js
├── app.js
└── calculator.test.js
```

index.html

HTML

```html
<!doctype html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Shopping Discount Calculator</title>
  <link rel="stylesheet" href="style.css">
  <script src="calculator.js" defer></script>
  <script src="app.js" defer></script>
</head>
<body>
  <main class="card">
    <h1>Shopping Discount Calculator</h1>
    <form id="checkout-form">
      <label for="price">Unit Price (BDT)</label>
      <input id="price" type="number" min="0" step="1" required value="1500">
      <label for="quantity">Quantity</label>
      <input id="quantity" type="number" min="1" max="100" step="1" required value="2">
      <label for="coupon">Coupon Code (optional)</label>
      <input id="coupon" type="text" maxlength="20" autocomplete="off" placeholder="SAVE10">
      <label class="check"><input id="member" type="checkbox"> Member (extra 5% off)</label>
      <button type="submit">Calculate Total</button>
    </form>
    <section aria-live="polite" id="result" role="status"></section>
  </main>
</body>
</html>
```

style.css

```css
* { box-sizing: border-box; }
body { margin: 0; min-height: 100vh; padding: 24px; background: #eef4ff; color: #172a46; font: 16px/1.6 system-ui, sans-serif; display: grid; place-items: center; }
.card { width: min(100%, 520px); padding: 28px; background: white; border: 1px solid #d9e4f5; border-radius: 16px; box-shadow: 0 10px 35px #17335c12; }
h1 { margin-top: 0; font-size: 1.5rem; }
form { display: grid; gap: 9px; }
input:not([type="checkbox"]) { width: 100%; padding: 11px; border: 1px solid #b9c8db; border-radius: 8px; font: inherit; }
.check { display: flex; align-items: center; gap: 8px; margin: 9px 0; }
button { background: #155bd5; color: white; border: 0; padding: 12px; border-radius: 8px; font: inherit; cursor: pointer; }
button:hover { background: #0d48af; }
#result { margin-top: 20px; }
#result.error { color: #a31730; }
.summary { margin: 0; display: grid; grid-template-columns: 1fr auto; gap: 7px 14px; }
.summary dt { font-weight: 500; }
.summary dd { margin: 0; text-align: right; }
.summary .grand { font-weight: 800; border-top: 1px solid #cbd8eb; padding-top: 8px; }
```

calculator.js — Pure Business Logic

```javascript
'use strict';

function calculateOrder({ price, quantity, coupon = '', isMember = false }) {
  if (!Number.isSafeInteger(price) || price < 0) {
    throw new RangeError('Price must be a non-negative integer (BDT).');
  }
  if (!Number.isInteger(quantity) || quantity < 1 || quantity > 100) {
    throw new RangeError('Quantity must be an integer from 1 to 100.');
  }
  if (typeof coupon !== 'string' || typeof isMember !== 'boolean') {
    throw new TypeError('Invalid coupon or membership value.');
  }

  const subtotal = price * quantity;
  if (!Number.isSafeInteger(subtotal)) {
    throw new RangeError('Subtotal exceeds safe integer range.');
  }

  const code = coupon.trim().toUpperCase();
  let couponRate = 0;
  switch (code) {
    case '': break;
    case 'SAVE10': couponRate = 0.10; break;
    case 'SAVE20':
      if (subtotal < 5000) throw new RangeError('SAVE20 requires BDT 5,000 subtotal.');
      couponRate = 0.20;
      break;
    default: throw new RangeError('Unknown coupon code.');
  }

  const couponDiscount = Math.round(subtotal * couponRate);
```

```
  const afterCoupon = subtotal - couponDiscount;
  const memberDiscount = isMember ? Math.round(afterCoupon * 0.05) : 0;
  const discountedTotal = afterCoupon - memberDiscount;
  const deliveryFee = discountedTotal >= 3000 ? 0 : 120;
  const grandTotal = discountedTotal + deliveryFee;

  return { subtotal, couponDiscount, memberDiscount, discountedTotal, deliveryFee, grandTotal };
}

if (typeof module !== 'undefined' && module.exports) {
  module.exports = { calculateOrder };
}
```

app.js — Form Events ও DOM Update

JAVASCRIPT

```
'use strict';

const form = document.querySelector('#checkout-form');
const result = document.querySelector('#result');
const money = value => `BDT ${value.toLocaleString('en-BD')}`;

form.addEventListener('submit', event => {
  event.preventDefault();
  result.replaceChildren();
  result.classList.remove('error');

  const priceText = document.querySelector('#price').value.trim();
  const quantityText = document.querySelector('#quantity').value.trim();
  const price = priceText === '' ? NaN : Number(priceText);
  const quantity = quantityText === '' ? NaN : Number(quantityText);

  try {
    const order = calculateOrder({
      price,
      quantity,
      coupon: document.querySelector('#coupon').value,
      isMember: document.querySelector('#member').checked
    });
    const rows = [
      ['Subtotal', order.subtotal],
      ['Coupon Discount', -order.couponDiscount],
      ['Member Discount', -order.memberDiscount],
      ['Delivery Fee', order.deliveryFee],
      ['Grand Total', order.grandTotal]
    ];
    const summary = document.createElement('dl');
    summary.className = 'summary';
    for (const [label, value] of rows) {
      const dt = document.createElement('dt');
      const dd = document.createElement('dd');
      dt.textContent = label;
      dd.textContent = money(value);
      if (label === 'Grand Total') {
        dt.className = 'grand';
        dd.className = 'grand';
      }
```

```
      summary.append(dt, dd);
    }
    result.append(summary);
  } catch (error) {
    result.classList.add('error');
    result.textContent = error.message;
  }
});
```

চালানোর নিয়ম: একই Folder-এ পাঁচটি File রেখে `index.html` Browser-এ Open করো। Calculator Library Pure Function হওয়ায় Browser ও Node.js—দুই জায়গায় Test করা যায়।

Manual Test Scenarios

Price	Qty	Coupon	Member	Expected Grand Total
1500	2	*(none)*	No	3000
1500	2	SAVE10	No	2820
2500	2	SAVE20	Yes	3800
1000	1	*(none)*	No	1120
1000	1	SAVE20	No	Error

Automated Tests

JAVASCRIPT

```
'use strict';
const assert = require('node:assert/strict');
const { calculateOrder } = require('./calculator');

assert.equal(calculateOrder({ price: 1500, quantity: 2 }).grandTotal, 3000);
assert.equal(calculateOrder({ price: 1500, quantity: 2, coupon: 'SAVE10' }).grandTotal, 2820);
assert.equal(calculateOrder({ price: 2500, quantity: 2, coupon: 'SAVE20', isMember: true
}).grandTotal, 3800);
assert.equal(calculateOrder({ price: 1000, quantity: 1 }).grandTotal, 1120);
assert.throws(() => calculateOrder({ price: 1000, quantity: 1, coupon: 'SAVE20' }), /requires/);
assert.throws(() => calculateOrder({ price: -1, quantity: 1 }), /Price/);
assert.throws(() => calculateOrder({ price: 1, quantity: 0 }), /Quantity/);
assert.throws(() => calculateOrder({ price: 1, quantity: 1, coupon: 'UNKNOWN' }), /Unknown/);
console.log('8 tests passed');
```

Terminal-এ `node calculator.test.js` চালাও। Browser UI-এর Automated E2E Test এই অধ্যায়ে অন্তর্ভুক্ত নয়।

৩.১২ অনুশীলনী

1. `7 % 3` এবং `-7 % 3`-এর ফল ব্যাখ্যা করো।
2. `0 || 50` ও `0 ?? 50`-এর ফল কী? কেন?

3. `if/else` দিয়ে Exam Grade নির্ধারণ করো: 80+ A, 60+ B, 40+ C, নইলে F। Invalid Score Reject করো।
4. `switch` দিয়ে `pending`, `paid`, `cancelled` Order Status-এর Label Return করো।
5. Project-এ `SAVE15` Coupon যোগ করো: Subtotal কমপক্ষে BDT 2,000 হলে ১৫% Discount।
6. `calculateOrder`-এ 0 Price এবং Quantity 1-এর জন্য Test লেখো।
7. Challenge: Discount Policy আলাদা Configuration Object-এ সরিয়ে Business Logic আরও Maintainable করো।

৩.১৩ অনুশীলনীর সমাধান

১. `7 % 3` → `1`; `-7 % 3` → `-1` । JavaScript `%` Remainder Operator।

২. `0 || 50` → `50` কারণ `0` Falsy; `0 ?? 50` → `0` কারণ `0` Nullish নয়।

৩.

```javascript
function grade(score) {
  if (!Number.isFinite(score) || score < 0 || score > 100) return 'Invalid';
  if (score >= 80) return 'A';
  if (score >= 60) return 'B';
  if (score >= 40) return 'C';
  return 'F';
}
```

৪.

```javascript
function statusLabel(status) {
  switch (status) {
    case 'pending': return 'Awaiting payment';
    case 'paid': return 'Payment complete';
    case 'cancelled': return 'Order cancelled';
    default: return 'Unknown status';
  }
}
```

৫. `switch`-এ `case 'SAVE15': if (subtotal < 2000) throw new RangeError(...); couponRate = 0.15; break;` যোগ করতে হবে।

৬. `assert.equal(calculateOrder({price: 0, quantity: 1}).grandTotal, 120);` ।

৭. `const couponPolicies = { SAVE10: {rate: 0.1, min: 0}, SAVE20: {rate: 0.2, min: 5000} };` রেখে Lookup ও Validation আলাদা Function-এ নেওয়া যায়। Object-এর Key Lookup করার আগে `Object.hasOwn()` দিয়ে Membership Check করা উচিত।

৩.১৪ MCQ Quiz

1. Strict Equality Operator কোনটি? A. `=` B. `==` C. `===` D. `=>`

2. `2 + 3 * 4` কত? A. 20 B. 14 C. 24 D. 9
3. `0 ?? 10` কত? A. 10 B. null C. 0 D. undefined
4. `5 === '5'` কী? A. true B. false C. 5 D. Error
5. `10 % 4` কত? A. 2 B. 2.5 C. 4 D. 0
6. কোনটি Logical AND? A. `||` B. `!` C. `&&` D. `??`
7. `switch`-এ Default Branch কোনটি? A. `else` B. `default` C. `otherwise` D. `fallback`
8. `Number.isNaN(NaN)` কী? A. true B. false C. 0 D. Error
9. Ternary Syntax কোনটি? A. `a ? b : c` B. `a : b ? c` C. `if ? else` D. `a ?? b : c`
10. Client-Side Price Calculation কেন Final Authority নয়? A. Slow B. User এটি পরিবর্তন করতে পারে C. JS Math পারে না D. HTML নেই

Answer Key ও ব্যাখ্যা: 1-C (Type ও Value দুটোই তুলনা), 2-B (Multiplication আগে), 3-C (0 Nullish নয়), 4-B (Type আলাদা), 5-A (Remainder 2), 6-C (AND), 7-B (Unmatched Case), 8-A (`NaN` সনাক্ত), 9-A (Condition ? True : False), 10-B (Client Input ও Code Manipulable)।

৩.১৫ Chapter Review Checklist

- ☐ Arithmetic ও Comparison Operators বুঝেছি
- ☐ `===` বনাম `==` ব্যাখ্যা করতে পারি
- ☐ `||` বনাম `??` সঠিকভাবে ব্যবহার করতে পারি
- ☐ `if/else`, `switch`, Ternary দিয়ে Decision নিতে পারি
- ☐ Shopping Discount Calculator চালিয়েছি
- ☐ Automated Tests চালিয়েছি
- ☐ Exercises ও Quiz শেষ করেছি

পরবর্তী অধ্যায়: Chapter 4 — Loops এবং Iteration।